Analysis of Transparency Levels in Distributed Database Management Systems (DDBMS)

Assignment Report

January 27, 2026

1 Introduction to Transparency

In a Distributed Database Management System (DDBMS), data is not kept in one place. It is spread across different computers, cities, or even countries. However, if a user had to remember which data is in which city, it would be very difficult to use.
Transparency is the "invisibility" of the system's physical details. It means that the user should feel like they are using a single, local database on their own computer. The main goal is to hide the complexity of the network from the end-user or the programmer.

2 Distribution Transparency

This is the most important level. It ensures that the user does not need to know how the data is physically distributed.

2.1 Location Transparency

With location transparency, a user can access any data without knowing where it is stored. For example, if a student at Techno Main Salt Lake wants to check their grades, they just write a query like *SELECT * FROM Grades*. They don't need to know if the "Grades" table is stored on a server in Kolkata or a server in Mumbai. The system automatically finds the address for them.

3 Advanced Distribution Levels

3.1 Fragmentation Transparency

In DDBMS, a single table is often broken into small pieces called "fragments."

- **Horizontal Fragmentation:** Splitting rows (e.g., rows for Kolkata students in one file, rows for Delhi students in another).
- **Vertical Fragmentation:** Splitting columns (e.g., Name and ID in one file, Fees and Marks in another).

Fragmentation Transparency means the user sees one whole table. They do not need to know the names of the fragments or how to join them back together.

3.2 Replication Transparency

Sometimes, the same data is copied (replicated) on multiple servers so that if one server fails, the data is still safe. **Replication Transparency** ensures that the user doesn't know these copies exist. When a user updates their phone number, the system updates all the copies in the background. The user only thinks they updated it once. This is very important for data reliability and speed.

4 Transaction Transparency

A transaction is a group of steps that must happen together (like a bank transfer). Transaction transparency ensures that everything stays correct even in a distributed network.

4.1 Concurrency Transparency

In a big system, thousands of people use the database at the same time. Concurrency transparency ensures that if two people try to update the same piece of data from different cities at the exact same second, the results are still correct. One person's work will not "overwrite" or "delete" the other person's work by mistake.

4.2 Failure Transparency

Network cables can break, and servers can crash. In a central database, if the server dies, everything stops. But in a DDBMS, if one server in the network fails, the system should keep running using other nodes. **Failure Transparency** means the user never even notices that a server went down. The system automatically redirects the request to a working server.

5 Performance and DBMS Transparency

5.1 Performance Transparency

The system must be fast. If a query is sent, the DDBMS must figure out the "Optimal Path." For example, if data is available in both Kolkata and London, and the user is in Kolkata, the system should pull it from the local server to save time. The user doesn't have to plan this; the system does it automatically.

5.2 DBMS Transparency (Heterogeneity)

In many companies, different sites use different software. Site A might use MySQL, while Site B uses Oracle. DBMS transparency allows the user to write a single query that works across both types of software. It "translates" the language so the user doesn't have to learn multiple database languages.

6 Conclusion

Transparency is what makes a Distributed Database usable. Without these levels, the system would be too complex for humans to manage. By hiding the location, fragments, copies, and hardware differences, a DDBMS provides a smooth and easy experience for everyone.

7 References

1. **M. Tamer Özsu & Patrick Valduriez.** *Principles of Distributed Database Systems.* Springer Science & Business Media.

2. **Ramez Elmasri & Shamkant Navathe.** *Fundamentals of Database Systems.* Pearson.

3. **Abraham Silberschatz, Henry F. Korth & S. Sudarshan.** *Database System Concepts.* McGraw-Hill Education.